The context window

A fixed amount of room

A model can only read a limited amount of text at once. That limit is the context window, and it is measured in tokens, where a token is about three quarters of a word. A short word is usually one token, while a rare or lengthy word is split into several. Punctuation and spaces count too. For ordinary English, a page of text is roughly 500 to 700 tokens.

Window sizes differ between models. Some hold a few thousand tokens and others hold hundreds of thousands. The window is a hard limit set by the model: a request that is too big is rejected, or the oldest part of it is cut off, depending on the provider.

What shares the window

The prompt, the conversation history, the tool results and the reply all share the window. It is one budget, not four. The instruction you write at the start uses some of it. Every message from the user and every reply from the model uses some more. Each tool description and each tool result takes its share, and so does the answer the model is about to write.

This matters because the reply needs room as well. If your input fills almost the whole window, the model has only a little space left to answer and its reply may be cut off in the middle of a sentence.

Why a model loses the start of a conversation

A model does not remember anything between requests. To give the impression of a continuous chat, your program sends the whole conversation again with every new message. So the history grows with each turn, and each turn costs more than the last.

When a lengthy conversation fills the window, the oldest messages must be dropped or summarised, so the model no longer knows what happened at the start. This is not a bug and it is not tiredness. The early messages are simply not part of the request any more, so as far as the model is concerned they never existed. If the user asks "what was my first question?" after the window has overflowed, the model cannot answer, because that text was never sent.

Programs handle this in a few ways. The simplest is to drop the oldest messages once the history passes a limit. Another is to ask the model to write a short summary of the old part, and keep the summary instead of the original messages. A third is to store important facts, such as the user's name or preferences, in a separate place and add them to every request. Each choice trades detail for room.

A useful exercise is to write down what a typical request in your own program contains, with a rough size for each part. Many people are surprised to find that the instruction and

the tool descriptions are already a good share of the window before the user has typed a word. Knowing the split tells you where to save room first.

Counting tokens

Because the limit is in tokens, not characters or words, programs need a way to count them. Each provider offers a tokenizer or a counting service, and there are open-source libraries that give close estimates. A rough rule is to divide the number of characters by four, but this is only a guide and it is less accurate for code, other languages and unusual text.

Counting also helps with cost. Providers charge by the token, usually at a different price for input and for output. A request with a large history costs more even when the new question is one line.

More room is not always better

It is tempting to paste everything into the prompt and rely on a large window. This has three problems. It is slow and expensive, because the whole text is processed on every request. It can make answers worse, because a passage that matters is surrounded by text that does not, and models can overlook details buried in the middle of a very large input. And it does not scale: your documents will eventually be bigger than any window.

Retrieval

Retrieval is one way to cope. Instead of pasting every document into the prompt, the program searches for the few passages that matter and sends only those. The model gets a small, focused set of facts, the request stays cheap, and the answer can point to the exact source. The documents can be far larger than the window, because only the relevant pieces ever enter it.

Retrieval works well together with a sensible history policy. Keep the recent conversation, summarise or drop the old part, and fetch fresh passages for each question. Every piece of text then earns its place in the window.

Summary

The context window is the model's whole working memory for one request, and everything competes for the space: instructions, history, tool results, retrieved passages and the reply. Measure it in tokens, plan for the answer to need room, and decide deliberately what to keep, what to shorten and what to fetch only when it is needed.